

Міністерство освіти і науки України Національний технічний
університет України "Київський політехнічний інститут імені
Ігоря Сікорського"

Фізико-технічний інститут

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №3

Криптоаналіз афінної біграмної підстановки

Виконали:

ФБ-33

Ольшевський Б.

ФБ-33 Степура Н.

Київ 2025

Мета роботи: набуття навичок частотного аналізу на прикладі розкриття моноалфавітної підстановки; опанування прийомами роботи в модулярній арифметиці.

Хід роботи

Варіант 5

1. Реалізувати підпрограми із необхідними математичними операціями: обчисленням оберненого елемента за модулем із використанням розширеного алгоритму Евкліда, розв'язуванням лінійних порівнянь. При розв'язуванні порівнянь потрібно коректно обробляти випадок із декількома розв'язками, повертаючи їх усі.

Код знаходиться в файлі 3lkr1.py

```
def extendedev(a, b):
    if b == 0:
        return a, 1, 0
    else:
        g, x1, y1 = extendedev(b, a % b)
        x = y1
        y = x1 - (a // b) * y1
        return g, x, y

def modin(a, m):
    g, x, y = extendedev(a, m)
    if g != 1:
        return -1
    else:
        return x % m

def solve_linear_congruence(a, b, m):
    g, x, y = extendedev(a, m)
    solutions = []

    if b % g != 0:
        return solutions
```

```

a //= g
b //= g
m //= g

inv = modin(a, m)
if inv == -1:
    return solutions

x0 = (inv * b) % m

for i in range(g):
    solutions.append((x0 + i * m) % m)

return solutions

def main():
    a, m = map(int, input("Введіть число та модуль для обчислення оберненого елемента (a m): ").split())
    inv = modin(a, m)
    if inv != -1:
        print(f"{a}^(-1) mod {m}: {inv}")
    else:
        print(f"Обернений елемент для {a} не існує за модулем {m}.")
    a, b, m = map(int, input("Введіть коефіцієнт a, праву частину b та модуль m для лінійного порівняння (a b m): ").split())
    solutions = solve_linear_congruence(a, b, m)
    if solutions:
        print(f"{a}x = {b} mod {m}: {' '.join(map(str, solutions))}")
    else:
        print(f"Розв'язку для {a}x = {b} mod {m} немає.")

if __name__ == "__main__":
    main()

```

Результат

```

hon/Python313/python.exe c:/Users/godro/OneDrive/Desktop/3lab/3lkri.py
Введіть число та модуль для обчислення оберненого елемента (a m): 38 5
38^(-1) mod 5: 2
Введіть коефіцієнт a, праву частину b та модуль m для лінійного порівняння (a b m):
4 3
2x = 4 mod 3: 2
PS C:\Users\godro\OneDrive\Desktop\3lab>

```

2 За допомогою програми обчислення частот біграм, яка написана в ході виконання комп'ютерного практикуму №1, знайти 5 найчастіших біграм запропонованого шифртексту (за варіантом).

Код	виконували	з	1	практикума
<pre>import re from collections import Counter, defaultdict import math import pandas as pd def calculate_entropy(values): total_count = sum(values) return -sum((count / total_count) * math.log2(count / total_count) for count in values) with open(' ', "r", encoding="utf-8") as file: my_text = file.read() my_text = re.sub(r'\s+', ' ', my_text) with_space = re.sub(r'^а-яА-Я ', '', my_text).lower() text_len = len(with_space) frequencies_with_spaces = Counter(with_space) bigram_counts_with_spaces = defaultdict(int) for i in range(0, text_len - 1, 2): bigram_counts_with_spaces[with_space[i:i+2]] += 1 bigram_counts_overlap_with_spaces = defaultdict(int) for i in range(text_len - 1): bigram_counts_overlap_with_spaces[with_space[i:i+2]] += 1 top_5_bigrams_with_spaces = Counter(bigram_counts_overlap_with_spaces).most_common(5)</pre>				

```

h1_with_spaces = calculate_entropy(frequencies_with_spaces.values())
h2_no_overlap_with_spaces = calculate_entropy(bigram_counts_with_spaces.values())
h2_overlap_with_spaces = calculate_entropy(bigram_counts_overlap_with_spaces.values())

with open("results_with_spaces.txt", "w", encoding="utf-8") as f:
    f.write("=== Результати для тексту з пробілами ===\n")
    f.write(f"Ентропія H1 (одиначні букви): {h1_with_spaces:.6f}\n")
    f.write(f"Ентропія H2 (біграми без перекриття): {h2_no_overlap_with_spaces:.6f}\n")
    f.write(f"Ентропія H2 (біграми з перекриттям): {h2_overlap_with_spaces:.6f}\n\n")

    f.write("=== Частоти букв ===\n")
    for letter, count in frequencies_with_spaces.items():
        freq = count / text_len
        f.write(f"{letter}: {freq} \n")

    f.write("\n=== 5 найпоширеніших біграм з пробілами ===\n")
    for bigram, count in top_5_bigrams_with_spaces:
        f.write(f"{bigram}: {count} \n")

bmatrix_with_spaces = pd.DataFrame(0.0, index=sorted(frequencies_with_spaces.keys()),
columns=sorted(frequencies_with_spaces.keys()))

sum_bigram_counts_overlap_with_spaces = sum(bigram_counts_overlap_with_spaces.values())
for bigram, freq in bigram_counts_overlap_with_spaces.items():
    bmatrix_with_spaces.at[bigram[0], bigram[1]] = freq / sum_bigram_counts_overlap_with_spaces

bmatrix_with_spaces.to_csv('bmatrix_with_spaces.csv')

mytext_no_spaces = re.sub(r'^a-яA-Я', '', my_text).lower()

mytext_len_no_spaces = len(mytext_no_spaces)

letter_frequencies_no_spaces = Counter(mytext_no_spaces)

bigram_counts_no_overlap_no_spaces = defaultdict(int)
for i in range(0, mytext_len_no_spaces - 1, 2):
    bigram_counts_no_overlap_no_spaces[mytext_no_spaces[i:i+2]] += 1

```

```
bigram_counts_overlap_no_spaces = defaultdict(int)
for i in range(mytext_len_no_spaces - 1):
    bigram_counts_overlap_no_spaces[mytext_no_spaces[i:i+2]] += 1
```

```
top_5_bigrams_no_spaces = Counter(bigram_counts_overlap_no_spaces).most_common(5)
```

```
h1_no_spaces = calculate_entropy(letter_frequencies_no_spaces.values())
h2_no_overlap_no_spaces = calculate_entropy(bigram_counts_no_overlap_no_spaces.values())
h2_overlap_no_spaces = calculate_entropy(bigram_counts_overlap_no_spaces.values())
```

```
with open("results_no_spaces.txt", "w", encoding="utf-8") as f:
    f.write("=== Результати для тексту без пробілів ===\n")
    f.write(f"Ентропія H1 (одиначні букви): {h1_no_spaces:.6f}\n")
    f.write(f"Ентропія H2 (біграми без перекриття): {h2_no_overlap_no_spaces:.6f}\n")
    f.write(f"Ентропія H2 (біграми з перекриттям): {h2_overlap_no_spaces:.6f}\n\n")
```

```
f.write("=== Частоти букв ===\n")
for letter, count in letter_frequencies_no_spaces.items():
    freq = count / mytext_len_no_spaces
    f.write(f"'{letter}': {freq}\n")
```

```
f.write("\n=== 5 найпоширеніших біграм без пробілів ===\n")
for bigram, count in top_5_bigrams_no_spaces:
    f.write(f"'{bigram}': {count} \n")
```

```
bmatrix_no_spaces = pd.DataFrame(0.0, index=sorted(letter_frequencies_no_spaces.keys()),
columns=sorted(letter_frequencies_no_spaces.keys()))
```

```
total_sum_bigram_counts_overlap_no_spaces = sum(bigram_counts_overlap_no_spaces.values())
for bigram, freq in bigram_counts_overlap_no_spaces.items():
    bmatrix_no_spaces.at[bigram[0], bigram[1]] = freq / total_sum_bigram_counts_overlap_no_spaces
```

```
bmatrix_no_spaces.to_csv('bmatrix_no_spaces.csv')
```

Результат

```
ктор/ср1/ср1.py
5 найчастіших біграм без пробілів:
вн: 56
тн: 51
дк: 50
ун: 50
хщ: 48
```

3-5

```
import re
from collections import Counter
from collections import defaultdict
import itertools
alh = "абвгдежзийклмнопрстуфхцчшщъыэюя"
m = len(alh)
m2 = m*m

def extendedev(a, b):
    if b == 0:
        return a, 1, 0
    else:
        g, x1, y1 = extendedev(b, a % b)
        x = y1
        y = x1 - (a // b) * y1
        return g, x, y

def mod(x, mod):
    x %= mod
    if x < 0:
        return x + mod
    return x

def modin(a, m):
    g, x, _ = extendedev(a, m)
    if g != 1:
        return -1
    else:
        return x % m
```



```
def solve_linear_congruence(a, b, m):
```

```
g, _, _ = extendedgcd(a, m)
```

```
solutions = []
```

```
if b % g != 0:
```

```
    return solutions
```

```
    a //= g
```

```
    b //= g
```

```
    m //= g
```

```
    inv = modinv(a, m)
```

```
    if inv == -1:
```

```
        return solutions
```

```
    x0 = (inv * b) % m
```

```
    for i in range(g):
```

```
        solutions.append((x0 + i * m) % m)
```

```
    return solutions
```

```
def read_ciphertext(file_path):
```

```
    with open(file_path, "r", encoding="utf-8") as file:
```

```
        return file.read().replace("\n", "")
```

```
def get_bigrams(text):
```

```
    bigrams = [text[i:i+2] for i in range(0, len(text) - 1, 2)]
```

```
    return Counter(bigrams)
```

```
def decrypt_text(ciphertext, inv_a, b):
```

```
    decrypted_text = ""
```

```
    for i in range(0, len(ciphertext)-1, 2):
```

```
        y = alph.index(ciphertext[i])*m + alph.index(ciphertext[i+1])
```

```
        x = (inv_a*(y-b))%m2
```

```
        decrypted_text += alph[x//m] + alph[x%m]
```

```
    return decrypted_text
```

```
def generate_candidates(common_bigrams, ciphertext_bigrams):
```

```

top_ciphertext_bigrams = [key for key, _ in sorted(ciphertext_bigrams.items(), key=lambda x: x[1],
reverse=True)[:5]]
candidates = set()
pairs = set()
for common_bigram in common_bigrams:
for ciphertext_bigram in top_ciphertext_bigrams:
pairs.add(((alh.index(common_bigram[0])*m + alh.index(common_bigram[1]))%m2,
(alh.index(ciphertext_bigram[0])*m + alh.index(ciphertext_bigram[1]))%m2))
for (x1, y1), (x2, y2) in itertools.combinations(pairs, 2):
all_a = solve_linear_congruence(mod(x1 - x2, m2), mod(y1 - y2, m2), m2)
for a in all_a:
ain = modin(a, m2)
if ain:
b = (y1 - a*x1)%m2
candidates.add((a, ain, b))
return candidates

def Index_C(text):
n = len(text)
if n <= 1:
return 0
frequencies = Counter(text)
ic = 0
for freq in frequencies.values():
ic += freq * (freq - 1)
return ic / (n * (n - 1))

def is_russian(text):
if Index_C(text) < 0.045:
return False
freqs = Counter(text)
common_letters = ["o", "a", "e"]
uncommon_letters = ["ф", "щ", "б"]

freq_of_common_letters = sum(freqs[letter] for letter in common_letters if letter in freqs) / len(text)
if freq_of_common_letters < 0.2:
return False

freq_of_uncommon_letters = sum(freqs[letter] for letter in uncommon_letters if letter in freqs) / len(text)
if freq_of_uncommon_letters > 0.05:
return False

```

```
popular_trigrams = {"про", "ста", "ено", "сна", "ног"}
trigrams = defaultdict(int)
```

```
for i in range(len(text) - 2):
    trigram = text[i:i + 3]
    trigrams[trigram] += 1
```

```
popular_trigrams_freq = sum(trigrams[t] for t in popular_trigrams)
popular_trigrams_freq /= sum(trigrams.values())
if popular_trigrams_freq < 0.0038:
    return False
return True
```

```
def decrypt_and_validate(ciphertext, common_bigrams):
    ciphertext_bigrams = get_bigrams(ciphertext)
    candidates = generate_candidates(common_bigrams, ciphertext_bigrams)
    for candidate in candidates:
        decrypted_text = decrypt_text(ciphertext, candidate[1], candidate[2])
        if is_russian(decrypted_text):
            print(f"Ключі знайдено а:{candidate[0]} б:{candidate[2]}")
            print("Перші 100 букв тексту:")
            print(decrypted_text[:100])
            with open("results.txt", "w", encoding="utf8") as file:
                file.write(decrypted_text)
            break
```

```
common_bigrams = ["ст", "но", "то", "на", "ен"]
```

```
ciphertext
read_ciphertext("C:/Users/godro/OneDrive/Desktop/3lab/05.txt")
```

```
ciphertext = re.sub(r'^а-яА-Я ', '', ciphertext.lower())
```

```
decrypt_and_validate(ciphertext, common_bigrams)
```

Функція `read_ciphertext` зчитує шифртекст із файлу, видаляє зайві символи й переводить текст у нижній регістр. Регулярний вираз видаляє всі символи, крім літер кирилиці. Функція `get_bigrams` ділить текст на біграми (послідовності двох літер) і підраховує їх частоти. Використовується функція `generate_candidates` для створення набору можливих ключів шифрування. Знаходяться найбільш поширені біграми в шифртексті та порівнюються з відомими частотними біграмами в російській мові (наприклад, "ст", "но", "то"). Для кожної пари біграм розв'язується система рівнянь (пошук лінійних конгруєнцій через функцію `solve_linear_congruence`). Отримуються можливі значення a (множник шифру) та b (зміщення), які формують кандидатів. Функція `decrypt_text` перетворює кожен біграм шифртексту у відкритий текст, використовуючи формулу афінного дешифрування:

$$X_i = a^{-1}(Y_i - b) \bmod m^2$$

Функція `is_russian` аналізує текст на відповідність російській мові. Обчислюється індекс відповідності (збіги між частотами літер). Перевіряються частоти найпоширеніших літер ("о", "а", "е") та рідковживаних ("ф", "щ", "ь"). Аналізуються популярні триграми (наприклад, "про", "ста")

І якщо знайдено ключ, текст дешифрується й зберігається у файл `results.txt`

Результат

```
PS C:\Users\godro\OneDrive\Desktop\3lab> & C:/Users/godro/AppData/Local/Programs/Python/Python313/python.exe c:/Users/godro/OneDrive/Desktop/3lab/lab3_1.py
Ключі знайдено a:654 b:777
Перші 100 букв тексту:
убивать больше не надо по слогам как он уже убил не следует ему быть благодарным иначе пришлось бы убивать самому это не
PS C:\Users\godro\OneDrive\Desktop\3lab>
```

Зашифрований текст

кеюибщаефдфмдкдролрццисвнуншвйняэшскевдтнюдаобсюсыэихзтмдълюхунхмъв
внсдуэммндтихкеюибщцызкзхшвнос

ютнйщтцншуссянхщлвжвпъкшвнмщзфтсхщпддкясввццтнавпгнуйввйнлхьерддыц
рихэкьзцэижцьехщмсэкжлрибуждэм

химьпьявсттнзцюсфспьзуйпдкнхркхульяцкчашьяншибжаксэкццзтцциюцншумщошья
щкщнфрхуюижсгцыззфрщихзтчщрихн

эпозтгфккчщкдмкльоьеынунййлцяьэрхнмкпмдкйпоиэуныэнсмнмсхэццьедктництнд
уццоэивупхюфйчсьивйэютнрцшэбв

щншуоздкдктнунянккфкьящиссбинкурдцбщдскрщянщкдкайищжшсвыьербщяяшнду
зйнкщнвнгоьцэииспытuumщщшдекхнду

аошдвдеигебуаявюсшъйдроццвнфиибжлакццвбвыавккслтьхцзйъцжъбрьецфтспьби
шиыовдъезбтнмсэкжлрчсхщърпшв
шнйьянсибжлтьчсйрьэчтнундулфтснсшбйнбжжцрнмющъккюиеуязэзтьяреурндуьц
оэгкмбобмцкскехюксдцтсывзтмсун
йьксщиссшнцзйъцйнпршьккфкяслркеййнавпъхсуншнузеумкжлакцисуьдьбкфипь
йнмсуншснхтуйнццмсяьмныюнцкрч
ыоклзфкчпвныуозрбжлжвцнхщсссцжъбипсрзфкаьихмнцэчсавозулбутнзцнулцзткоц
цвнфиибхюпвиэислбиювинхыршыв
цнярбщфджлзйъцйнзцнулцяьйнвнцхркпрыожврщьянкиюдждкеспыбубиюхщбуакикя
еэдакаоццсвлбеилрлвцофкяышвнун
хщлвэкжлтьосцнхщиютнуншнмстспльаихщрнньнхшвщшвносчсабъешижсоэосыумц
мбриввудябакфурщяэлчяздкаийечслс
осэкццяьцнэлязаяьцнхщсссцжъьзжлмщунавшьавзтьяюсуйвнакдуюиюьяучмпрфдййвди
хрнфззфтнхщхиеуязэзтьяюуццыьбь
еелфеипвидийдкаязщпупзобчсуьвнлвмьтнщъеэдвнстйндуаомнщоццвнфиибхюихтоц
цсввныклрынпьювюсисцйвнихщлр
акоющъцнхщбщщйтннсхщдкищъешичщкздукчввзтьяакккйдищжлывьктзихывуллвовя
вшньйссцпрыоынчкццяьклхнцэюдри
исэкжллреуныктзшрэчшиязиебчлвацлотнуншнмстспыцшэмвшщкзлаябсчбщщдыцэ
икзясусйнюйозвътныэакосжцшншвюи
йдьашншвосюсчязиьсунуллвихывхдскклмщубшскуаохщрнрцязакубсчфкяяосгйрщтнг
бфдзйъцэибусчжвавмнззфдыюиюшс
осюдритьйьнсхщтньцмрнннстрсосуллвзтвднкцяубщхичщмщтсчтгнэкхуямйдчщццм
нрншвйнвлвацшвхаврщшницюиьс
щожсюдгнуцрнчзшрынулцхдвмыцнрнуьнцяедьхсцнфуэюосйсчцэидктнуншнмншспьч
швнюдцфвдыюияосунйпщнбкчзиввнмн
рьнсибчзлориисэибудкяспнззжлфсчсбкаышнтныьзтпэпымвзтьсйядуццщццспрчсэьлв
зтклбулцшвюибщыцвивнуйвнакеи
чмывпвыэдчфкклццсвынуняуумпъшврцциссцмющиюлврлиэйбдцриьцяьввюдаолы
фьмодкчьяуфкойнкйдлцыцтнавчзфдыю
жяшсввдуюиэбывщшвныэлыдыщубшврчязрщвдойвнвнмщнсунцомюхщньюссттнхщщ
щфддбтьпнзкьеэдхнщъжвзтфрлцдкаяхь
овюсстхщрнпъйнщофкпрынсиульдццхифсчсхдййрснсерццисшнюсшьсцклтьпвидрош
ифкяяшнюдаоосунчзфпыцэилцмяэьсц
клжшвнунакубакюйтносшнпьявыйвшнщожсунюэсцэиринкгеэдвэцнпдрщрнчстнввшпв
пыызмбйнвнцхпнуцязьсйядуулрибу
вдвнщозыгйбчйдсчбщиэбкдктнхщхилвннюсвнщокнирэчрнианцяеьцтсывзтосибфддбп

мьлриввееяхэфртгрулцузбщшьав
тулцибсчннисозфдыюжлрдцбщшдскрщиэбквэгвжвзтшвжъаоеитншнпвихэхаорщибяс
фсчсщъавпъскгыюющлхвииспъвиул
бутнзцнулцьяжцюсчвввийимюгвшнщиющюирсунлсгоьрыноьхоцвнфиибкзенуьпъбцрн
ыгщйеуйнзщшьавхщеуеидебупьесуз
ющдкясюэсцэиьцзтнмслдроавежбщяйрщйуюйлцеищъккфдкфьнхчщмщявисчтжъа
маофисрябсчшижслбубщэнцфдэмсщябуб
чзйсанэирщхщмсэктзлэусхщрнляпдгсгцшфдкфьввнкубубяслоюищщшдекщсхдскхсов
пннчубакакхуямдкяхсвнхбжсмкщн
щъжвэкссщъккдктнфифсбвбдкястнтнмслдъшсвщйьшнсиеуюкыщцспрыьлнфкйдщщ
зйьцйныэвнхбрифкййунрншьвнбкубье
бчсвйнжндуеисхавупмююсшодкльулбусчцнннстрсшншвъхаврщянсцознкссьеуснсмнм
снсибссвддцйнчсщнэпозцфибссщщ
убссвнхбрифкясхщфдцьяклрыоибсчфкщйвносэиэчпнзкцьяклакаолржцяьзтхдицфпт
нхщыгложфьцэидктнунэибунсхщав
ьвлващеутнищлрдцбщшдщйьнвнцхдздкицмьяхавьщвуцфьцжыщнмкпмдкяярнэирщвв
пноулцфрынщхыщмснфжврйвньркзскыщ
ссвнхбрифкясозййцфцнюириьсосйгыовдриклакязеудкяюсузмщчявввнищрилващшвь
ичдрщдкикгбмщбуцстссьйшьвоейу
лцгйщщфкнхдкбщщйвнихобсчшибщекбщэюнхзциссичщиютнмслдфишдмбццмсгцшвэ
рзфвджяжвявшнмсчярщхьовюстымцкзищ
ссыршьудццрреулфщщаефдхссиroyяьисшщкзпксчролвтнрицнмскмжявзтсиюгщхт
нмспбмщбуцськмюннисдкдкцфжвьдт
мщшвпвкмжяьмщшвжърефщакиеэдакролфбклцбуябзщбукзунгэщъккгнввшнивжврщ
рныуоззнбкжлтьбцрныгйснжшдекцгеэ
юрсхщньбиулбунхнчйдпнввкцйнуншвъэтнщоьцсуюьсцтгуьйнньосфипьявпъпршьйлх
авьщсиеуобмбмщбуцсфрмщчяовуп
мюосшнкуаохщмсэкццзтбьыьмнжннуыфрыэиьсфсчсщъавозщсосгйлцмктзулынйнуа
ихщавиэжъчцоуобмблвыьрнунокпмшр
дцбщшддбубихйсансцрбжлвэкхюдрошджсюсунынмсйкмбкзхщхурсунщхвввмдкорыу
снчъьяуиюшсвпнкурмщеувирсунсцць
блшэннбвामозмщбвскаьшнжъжвупклэчйдищьешиивебпрябакоьзтянщиссийебчввтсзк
иющъккбыюскчицпьявицчзивьяочлц
свпдгсуфдкфьяэюдаорибщвчрытнрсбидуаодункющхихьсхдгсунфрлцдкяакдункчзжс
юсбчкнбквьфзтнуноьюддкнхживнал
буыодкеиочоьлхэфдкфьпылннсвнмкхсмщтсывзтьятнафкпрябйожсюсунюиикцфтсвв
щбакксйнбжрисцвджцмнщъкмыгьяе

хщсяюсстхщрнхщбщыцвиклаккзеуцнсюсияоусчтсьзтклрццюсстшнюдкшвнгьерыннь
эынавэкиютыннькиютнобакеишдщщ
швпвмндтихжцшнйнюирсыэьяокпмаобщцсэщбушсхщмсэкссейпфкясищхнэкмбжлжв
ннстрсосщэтсъяубщыцввяфжсюсунтс
чтгвмьввьелвмкроеэзтдццрнмюхщбуакдожсвнйсзвпфхщчсъязтьяйкчзфсчсгэлнцн
ерссжофкеиябпвистнпвюскиосыр
ынщэгожсгцмефдфмжяосзкццзтпытнрсакьлмщриарзфеуэирибщхиьсуйвнихвнстйнян
цуфкщщцсунхдицяедьакхуумжсвнчр
лвнзьтъяйкчзезьцюсжрыщумыцэиясезьцвнвнунищьеяцпьерынхщщыцвиьянсибяшн
лсиьпвтснфюирыюсцяккнивжошижс
мкарссжозщццесшндцнсккаирсыэокпмщнввйкриаршьлнуьэиулбунхмокздцрнфзфпдк
яснпчкхуцфюижсщщязюсшсиэжьввш
язосрнеелююисьфиосэщублыунчяюэецзивьяюкхуямщщшдбофдгвмсжкддьяжъяуцнв
ввшнмьвврщозенйсуньейпфкаьтныо
еущькхзцнулцзтднчелвпгцбуавкмлыкльтяуаишдщцмюкеоубщыцвиакэмлхчярщтсчт
рьйнвнцхмьакггмщшджсунлххэхьзт
лрэчбудкввзнвшнжьжврщунынжвжрццисчцэиаьмчвврщищсскжэжвмндтфрлцяьклх
нгцязвэкьзцэиьшсвмдьцюяусиебчду
ьешдриезмщюиоуриесввхьовэкжятнмслдзьлсрщйносыклрлврнвлэусхщрнавпгбубс
вйнавдьоспншсмкпрынкчмсхщнкой
щщбщшдмефдфмжлрифсбвбддкяяюввйнщцыгевввиймэоьжйвнакеиэчпидфккнйкр
ижэпншнхцынгспнунрнгошддкяяфсшью
арфдрижлццэчсавпзншвйнрнкизфтсиспнкгбмщбуцссцшнмьввьщянмсхмдктнянкк
бщшдекццжлыивквэпншнхцынгспныэ
рнгошддкйыавзтцнюфввовявлиьцяьокпмаишнмнээхфкччтхдицивьспьгсунмщпвюдцф
юирыусунлрлцдкяяюаокнввпфзлц
внстбвхщщслэмдчзоулыфьтгложфьцэидкнхпынкчмстспьвифщгбрыяьцщжлзфпреур
ндцвныкмбарбуябакфккчявпвлсзврщ
ьяшнынйнмьунжиюхщлвхщпэжвчспьпрццсвпддктндклцнулцмклытсющшдекццзтиэя
рчсжвюсстибдцнътсюсстхщээрщьечщ
кзмщрнтслкеурьомюхщньюссттнулбуввзнтснфчзццзтвииярщьякбньависйщкзхщхуи
юшннуаетнхщюиафккчлспьюпърц
мнрншбынлсюдризьяуфкшдвчсксчавзтрщхсщв

Розшифрований текст

убивать больше не надо после того как он уже убил, но следует ему быть благодарным, иначе пришлось бы убивать самому. Это не однолишь доброе страдание, это отождествление на основании одинаковых импульсов кубийству собственное, говоря лишь в минимальной степени смещенный нарциссизм. Этическая ценность этой доброты этим неоспаривается, может быть это вообще механизм нашего доброго участия по отношению к другому человеку, особенная просто упавший в чрезвычайном случае обремененного сознания своей вины писателя нет сомнения, что эта симпатия по причине отождествления решительно определила выбор материала для Достоевского. Но сначала он из эгоистических побуждений выводил бы кновенного преступника политического и религиозного, прежде чем к концу своей жизни вернуться как первопреступник, у которого убийца и сделать великое свое поэтическое признание и опубликование его посмертно. Но наследия и дневников его жены ярко осветило один эпизод его жизни, то время когда Достоевский в Германии был обуреваем горной страстью, Достоевский зарулет кой-явный припадок патологической страсти, который не поддается иной оценке, ни с какой стороны, не было недостатка в оправданиях этого странного и недостойного поведения, чувств, вины, как это нередко бывает у невротиков, нашло конкретную замену обремененности долгами и Достоевский мог отговариваться тем, что он привык играть и получил бы возможность вернуться в Россию, избежав заключения в тюрьму, кредиторам, но это было только предлог. Достоевский был достаточно проницателен, чтобы это понять, достаточно честен, чтобы в этом признаться, он знал, что главным был азарт, а сама по себе все подробности его обусловленного первичными позывами безрассудного поведения служат тому, чтобы доказать, что есть нечто, чему он не успокаивался, покаяние, а не все, что он играл, для него так же средством наказания, не считая количество, раздавал он молодой жене слово или личное слово, больше не играть, или не играть в этот день, он нарушал это слово, как он рассказывает почти всегда, если он своими проигрышами доводил себя и ее до крайнего бедственного положения, это служило для него еще одним патологическим удовлетворением, он мог перед ней поносить и унижать себя, просить ее презирать его, рассказывать, а что она вышла замуж за него, старого грешника и после всей этой разгрузки совесть, наследующий день игра начиналась снова, и молодая жена привыкла к этому циклу, так как заметила, что от этого действительно только можно было ожидать спасения писателя, и он никогда не продвигалось вперед, лучше чем после потерь всего из-за складывания последнего имущества, связав всего этого, она конечно не понимала, когда его чувство вины было удовлетворено, наказаниями, которыми он сам себя приговорил, тогда исчезла затрудненность в работе, тогда он позволял себе сделать несколько шагов на пути к успеху, рассматривая рассказ более молодого писателя, нетрудно угадать, какие давно позабытые детские переживания находят проявления в горной страсти, у Стефана Цвейга, посвятившего между прочим Достоевскому один из своих очерков, три мастера в сборнике, смятение чувств, новое, двадцать четыре часа в жизни женщины, этот маленький шедевр показывает, как будто лишь то, как им безответственным существом является женщина, и насколько удивительные для нее самой нарушения ее толкает неожиданное, ее жизненное впечатление, и новое, лаэа, если подвергнуть ее психоаналитическому толкованию, говорит, однако, без такой оправдывающей тенденции, и гораздо больше показывает, что все, что оно вообще человеческое, и, скорее, вообще мужское, и такое толкование, не столько явно, но сказано, что нет возможности, и его не допустить для сущности художественного творчества.

рактерно что писательского торым меня связывают дружеские отношения в ответ на мои расспросы утверждал что упомянутое толкование ему чудно и во всем не входило в его намерения не смотря на то что в рассказе вплетены некоторые детали как бы рассчитанные на то чтобы указывать на тайный след в этой новелле великосветская пожилая дама поверяет писателю то что ей пришлось пережить более двадцати лет тому назад рано овдовевшая мать двух сыновей некоторые из них более не нуждались от казавшаяся от каких бы то ни было надежд на сорок втором году жизни она попадает в во время одного из своих бесцельных путешествий в игорный зал монакского казино где среди всех диковин ее внимание привлекают дверуки которые еспотрясающей непосредственностью и силой отражают все переживаемые несчастными игроками чувства руки эти руки красивого юноши писатель как бы без всякого умысла делает его ровесником старшего сына наблюдающей за игрой женщины потерявшего все и в глубочайшей отчаянии покидающего зал чтобы в парке покончить с своею безнадёжной жизнью и не зная симпатии заставляющей женщину следовать за юношей и предпринять все для его спасения он принимает ее за одну из многочисленных в том городе навязчивых женщин и хочет от нее отделиться но она не покидает его и вынуждена в конце концов в силу сложившихся обстоятельств стать с ним его номере отеля и разделить его постель после этой импровизированной любовной ночи она велит казаться бы успокоившемуся юноше дать ей торжественное обещание что он никогда больше не будет играть с ним и дает ему деньгами обратный путь с своей стороны давая обещание встретиться с ним передуходом поезда на вокзал и затем в ней пробуждается большая нежность к юноше она готова пожертвовать всем чтобы только сохранить его для себя и она решает отпустить его с ним вместе в путешествие в место того чтобы с ним проститься всячески и помехи задерживают ее и она опаздывает на поезд то скепоисчезнувшему юноше она снова приходит в игорный дом и с возмущением обнаруживает там те же руки на кануне возбуждавшие в ней такую горячую симпатию нарушитель долга вернулся и игре она напоминает ему об его обещании но он держимый страстью он бранит сорвавшую его и грувелителей убраться вон и швыряет деньги которым она хотела его выкупить опозоренная она покидает город впоследствии узнает что ей удалось спасти его от самоубийства эта блестящая и без пробелов мотивировка написанной новеллы имеет конечно право на существование как таковая и не может не произвести на читателя бо льшого впечатления однако психоанализ учит что она возникла на основе умопостроения вождления периода полового созревания юкаковом вождлении и некоторые вспоминают совет ршенно сознательно согласную умопостроению вождлению мать должна сама ввести юношув половую жизнь для спасения его от заслуживающего опасения вреда она измастольчасты есублимирующие художественные произведения вытекают из того же первоисточника поро конанизма замещается пороком игорной страсти ударение поставлено на страстную деятельность рук предательски свидетельствует об этом отводе энергии действительно игорная держимость является эквивалентом старой потребности и она изменилась одним словом кроме лова и гранельзя называть ее аа

Висновок

Під час виконання цієї роботи були набуті навички застосування частотного аналізу для розкриття моноалфавітної підстановки. Було розглянуто методику визначення ключа для дешифрування шифртексту за допомогою принципів модульної арифметики.